Mock PS: Quadrotor Single Motor Failure

Overview

Problem: keep a standard quadrotor under control after one of its four motors fails.

Deadline: Oct 16, 2026 , 6:00 PM IST.

> **How you will be judged.** You will not be judged on whether you fully solve the PS or on how good your results are. Your main evaluation is your **mathematical understanding**: of the concepts, the algorithms you chose, the parameters you tuned and why, and the hypotheses you tested. A partial solution with clear, correct reasoning will score higher than a working result you cannot explain.

Who should attempt it: anyone comfortable with Linux and Python or C++. You do not need prior drone experience. Controls, robotics, ML, embedded and hardware people are all welcome; the tasks are split so each can show their strength.

Part	What you do	Points
Step 0	Set up PX4, Gazebo and MuJoCo	Required, not scored
Task 1	Research past solutions and papers, write the maths, run baseline benchmarks	65
Task 2	Build a reproducible fault injection and detection pipeline in PX4 SITL	35
Optional	Track A (KPIs and optimisation) or Track B (hardware sizing)	up to 20
Bonus	Startups and open code in this field, good documentation, and simulation vs real-world analysis	extra credit

What the problem looks like. A quadrotor has four motors. When one stops, the drone loses the ability to control all three rotations at once. The standard fix is to give up yaw: let the drone spin about its vertical axis and use the remaining three motors to control tilt

and height. Your job is to detect the failure quickly, switch to a controller that does this, and keep the drone in the air.

Rules and submission format

Rules

- You may use any public code or paper, but you **must cite it** and say exactly what you changed.
- AI tools are allowed. But if you cannot explain your own maths, code or results in the interview, you will be heavily penalised, up to zero for that task.
- Same physical setup as the real PS: standard quadrotor, onboard sensors only (IMU, magnetometer, barometer, GPS), no reversible props, no tilting or servo mechanisms, PX4 stack.

What to submit. No GitHub needed. Upload everything to one Google Drive folder, set sharing to "anyone with the link can view", and submit that Drive link through the Google Form. The folder should contain your main document (PDF, Word, Google Doc, Notion page or Claude artifact link) and:

1. **Your report:** maths, logic, pipeline, assumptions and results, organised by the task numbers of this PS.
2. **Code:** attached files, a shared Drive folder, or links. Any form is fine as long as we can open it.
3. **Graphs and results:** plots and tables, with labels and units.
4. **Logs:** PX4 `.ulg` files (or a link to them) behind your results.
5. **Videos:** simulation recordings, GIFs or stills.

AI-generated or Claude-generated documents, artifacts and code are all accepted, **as long as you can explain every part of them** in the interview.

Reproducibility matters. A reviewer may try to rerun your work, so include the steps, code and logs needed to do it.

Step 0: Prerequisites (required, not scored)

Set up the following on **Ubuntu 22.04/24.0/26.0 /Windows/MAC with docker or WSL** yourself, using the official documentation listed in References. Figuring out the setup is part of the test. Put a screenshot of each working tool in your report appendix.

1. **PX4 Autopilot** built for software-in-the-loop (SITL) simulation.
2. **Gazebo** (Harmonic, or Classic if you reuse 13.0 code) running a PX4 quadrotor that can take off and land.
3. **QGroundControl** to fly the simulated drone and change parameters.
4. **MuJoCo** with a quadrotor model, for fast testing of maths and controllers.
5. **Log tools** to read and plot PX4 `.ulg` flight logs.
6. **Motor failure injection** in SITL: make one motor stop mid-flight and watch the stock PX4 controller fail. That crash is your starting point.
7. *(Optional)* **ROS 2** with the PX4 bridge, if you want to write your code as ROS 2 nodes. (with is not recommended)

Task 1: Research, maths and baseline benchmarks (50 points)

Goal: understand what has already been done, explain the maths in your own words, and prove it with a small simulation anyone can rerun.

1a. What did the 13.0 teams do? (10)

Study at least **four** public Inter IIT 13.0 solutions (see References).

For each team, cover these points (a table in your report is fine):

- **Team** and institute
- **Fault injection method** they used
- **Detection method and latency**
- **Controller** type
- **Tiers reached:** controlled landing, hover, or return to home
- **Hardware:** did they fly it on a real drone?

Then answer: which detection signal did each use (angular acceleration, attitude error, ESC RPM, ML on IMU), and which controller (PID with yaw given up, LQR, INDI, NMPC, RL)?

1b. Literature and maths (20)

Read at least **five** papers (see References). Then write, in your own notation:

1. **The quadrotor model.** Position, velocity, attitude and body rates, with thrust and torques from the four rotors:

```
m\ddot{\mathbf{p}} = R\,\begin{bmatrix}0\\0\\\textstyle\sum_i
f_i\end{bmatrix} - m g\,\mathbf{e}_3, \qquad J\dot{\boldsymbol\omega} =
\boldsymbol\tau - \boldsymbol\omega \times J\boldsymbol\omega
```

2. **The control allocation matrix** that maps rotor thrusts to total thrust and torques, and what happens to it when one column (one rotor) becomes zero.
3. **Why yaw must be given up.** Show that with three rotors you cannot make total yaw torque zero while also balancing roll and pitch, so the drone settles into a steady spin. Explain the "relaxed hover" or "primary axis" idea from Mueller and D'Andrea (ICRA 2014).
4. **The equilibrium.** Compute the spin rate and rotor thrusts at relaxed hover for a given mass, inertia and arm length.
5. **Detection maths.** Pick one method (residual observer, angular-acceleration threshold, EKF, or learned classifier) and write its equations and decision rule.

1c. Gaps in 13.0 and limits of existing work (15)

Write one page on what was missing. Points to consider: detection only tested at low speed, no root-cause diagnosis (dead motor vs lost propeller vs weak motor), default Gazebo parameters instead of real ones, no `.ulg` logs, controllers tested only in hover, no real flight from most teams, and computation too heavy for the flight controller (for example NMPC). Back each point with evidence from a repo or paper.

1d. Baseline benchmark (20)

Build a **small, reproducible simulation** in Python (NumPy or MuJoCo), MATLAB/Simulink, Julia, CasADi, AMPL or any open-source tool. It must:

- simulate the quadrotor, cut one rotor at a chosen time, and run at least **two** approaches (for example: stock PID that crashes vs a yaw-free controller; or INDI vs LQR);
- report the same KPIs for each: altitude loss, maximum tilt, horizontal drift, time to recover, and spin rate;
- include plots of position, attitude, body rates and rotor thrusts before and after the failure.

A one-command run (`python run_benchmark.py` or `run_benchmark.m`) that reproduces every plot is expected.

Task 2: Fault injection and detection pipeline (35 points)

Goal: a pipeline anyone can run that injects motor faults in PX4 SITL, detects them, and logs everything in `.ulg` files. You may start from public 13.0 code; say what you reused and what you improved.

2a. Fault injection (8)

Inject faults on **any of the four motors**, at a chosen time, from a script (no clicking). Support at least three fault models:

Fault model	What it means	How to simulate
Complete stop	Motor or ESC dies	Motor output forced to zero
Propeller loss	Motor spins, no thrust	Thrust coefficient set to zero, RPM stays high
Partial loss	Weak or damaged motor	Thrust scaled to 30 to 80 percent
Intermittent (optional)	Loose connector	Output toggles on and off

2b. Detection (12)

Your detector must report **that** a failure happened and **which** motor failed. Bonus marks if it also says **why** (stop vs prop loss vs partial), because the real 15.0 PS scores root cause.

Run it in any **flight phases**: hover, forward flight, climb or descent, and yaw rotation. Use the IMU only, or IMU plus ESC RPM if you justify it.

2c. Benchmark (15)

Run the flight phases and your fault model, with random fault timing. Choose and justify your own metrics; for example:

Metric	Definition
Detection rate	Faults detected / faults injected

Metric	Definition
False alarm rate	False detections per minute of healthy flight (fly at least 10 healthy minutes)
Detection latency	Time from injection to detection, mean and worst case, in ms
Isolation accuracy	Correct motor identified / faults detected
Root-cause accuracy	Correct fault type / faults detected (if attempted)

Show a plot of latency vs flight phase, and a confusion matrix for isolation. Then tune one thing (threshold, filter, model) and show the before and after numbers.

Submit the `.ulg` logs for at least one run per fault model, and in each log mark the injection time and the detection time (for example as a custom uORB topic or a log message).

Optional tracks (up to 20 points, pick one)

Track A: KPIs and optimisation

1. **Define the KPIs** precisely, with formulas: altitude loss, attitude (tilt) error, horizontal drift, time to recover, spin rate, peak motor saturation, energy used, and compute cost (time per control step on a Pixhawk-class STM32H7). Find at least two more KPIs from papers.
2. **Optimise** your Task 3 controller against these KPIs with one or more methods: grid or Bayesian search over gains, CMA-ES, RL (PPO or SAC), imitation learning from an expert controller, or statistical tuning (for example z-scores or CUSUM for detection thresholds).
3. **Show the trade-offs** with a Pareto plot (for example drift vs altitude loss, or detection latency vs false alarms) and say which setting you would fly and why.

Track B: Hardware sizing

Design a quadrotor that can actually fly on three motors, at the **lowest cost**.

1. **Physics first.** At relaxed hover with one motor dead, the motor opposite the failed one runs near idle, so the remaining two carry most of the weight. So each motor must be able to produce at least half the vehicle weight, with margin for control: aim for a total

thrust-to-weight ratio of **at least 2.5 to 3** (normal drones use about 2).

2. **Calculate**, with reasoning:
   - total mass budget and the inertia matrix (treat motors as point masses on the arms, body as a box);
   - arm length (longer arms give more torque, but more inertia and weight);
   - required thrust per motor, then propeller size, motor KV and battery cell count, using manufacturer thrust tables;
   - ESC current rating with margin (the two loaded motors run near full throttle);
   - expected spin rate at relaxed hover, and check it fits the gyro range (most IMUs saturate at 2000 deg/s, about 35 rad/s);
   - flight time on three motors.
3. **Component list with prices**: frame, motors, propellers, ESCs (prefer ones with RPM telemetry, for detection), flight controller (PX4-compatible, for example Pixhawk 6C), GPS, battery, charger, power module, radio, telemetry, safety tether or net. Give a total cost and one cheaper alternative.
4. **Test plan**: how to test safely (tethered first, low altitude, kill switch).

Bonus: who works on this

Bonus: Controller for stable hover on three motors

Goal: after the failure is detected, keep the drone hovering in place, spinning but not falling or drifting away.

What to build. Any approach is fine if you justify it: PID with yaw given up, LQR around the relaxed-hover point, INDI, NMPC, geometric control, or a learned policy (RL or imitation). You may build on 13.0 code or published code (for example the uav_ftc trained policy); show what you added.

Show us:

1. **A video** (screen recording of Gazebo or MuJoCo, 30 to 90 s) showing takeoff, the failure, recovery and hover. Stills or GIFs in the report too.
2. Plots of position, attitude, body rates and motor outputs, from the `.ulg` log.
3. One paragraph on how you would make it fly on real hardware (what would break first).

Bonus: List companies and startups

List startups, companies, labs and people working on fault-tolerant multirotors, and all open-source code you can find. For each: name, what they do, link, and whether code or videos are public. Starting points: Inter IIT 13.0 teams (IIT Kharagpur ARK, IIT Kanpur, IIT BHU), UZH Robotics and Perception Group, TU Delft (Sihao Sun), HIT Shenzhen, Zhejiang University FAST Lab, DLR, PX4's own motor failure work, and drone safety startups in India and abroad.

Bonus: good documentation

Extra credit for a submission that someone new could pick up and use: a clear structure, every figure labelled and explained, equations defined before they are used, a short glossary, and setup and run steps that actually work. Good documentation is what turns your work into something the team can build on.

Bonus: simulation vs real world

Simulation is cleaner than reality. Extra credit if you show **where the gap is** and how it would affect your solution. For example:

- add realistic effects to your simulation (sensor noise and bias, IMU vibration, motor response delay, thrust curve mismatch, battery voltage drop, wind gusts, mass or inertia error) and show how your detector and controller degrade;
- compare your simulation against real flight data, such as public PX4 logs (Flight Review, UAV-SEAD) or published real-flight results, and point out the deviations;
- identify which assumption in your maths breaks first on a real drone, and why.

Show it with plots, numbers or logs, not only words.

How we grade

Criterion	What earns marks
Understanding	You explain the maths in your own words and can answer questions on it
Evidence	Every claim is backed by a plot, a log, a number or a citation

Criterion	What earns marks
Reproducibility	A reviewer can run your README and get your results
Honesty	You say clearly what failed, what you reused and what is yours
Clarity	Short, well-organised report; plots have labels and units

A small, working, honest result scores higher than a large, untested one.

References

Problem statements

- Inter IIT 13.0 ideaForge PS (the 15.0 PS has the same task with a new rubric)

Inter IIT 13.0 public solutions

- IIT Kanpur Team 30: PX4 detection module, NMPC, reports, videos
- Aerial Robotics IITK: NMPC and INDI
- IIT Gandhinagar: ROS 2 detector and controller
- Team 67: threshold and MLP detection, NMPC and PPO
- FaultTolerantControl-PX4: single PX4 module, jerk-based detection
- IIT Kharagpur ARK (13.0 Gold, method description only)
- px4-motor-failure-simulation: clean failure injection through the PX4 control allocator

Papers

- Mueller and D'Andrea, Stability and control of a quadrocopter despite the complete loss of one, two, or three propellers, ICRA 2014
- Sun et al., High-speed flight of quadrotor despite loss of single rotor, IEEE RA-L 2018 (INDI)
- Nan, Sun, Foehn, Scaramuzza, Nonlinear MPC for quadrotor fault-tolerant control, IEEE RA-L 2022
- Beyer et al., Incremental passive fault-tolerant control for quadrotors subjected to complete rotor failures, JGCD 2023

- Chen et al., Learning-based passive fault-tolerant control of a quadrotor with rotor failure, IROS 2025 (code)
- Zhou et al., Rotor-failure-aware quadrotors flight in unknown environments, IEEE TAES 2026 (videos)

Research code

- uzh-rpg/fault_tolerant_control
- SihaoSun/INDI_Quadrotor_FTC (MATLAB/Simulink)
- iff-gsc/Passive-FTC-Quadrotor

Docs and tools

- PX4 docs: SITL, failure injection, control allocation, log analysis (docs.px4.io)
- MuJoCo Menagerie for quadrotor models
- pyulog, PlotJuggler and review.px4.io for `.ulg` logs

Setup documentation (for Step 0)

- PX4 development environment on Ubuntu
- PX4 Gazebo simulation and Gazebo Classic simulation
- Gazebo Sim documentation
- QGroundControl user guide
- PX4 system failure injection
- PX4 control allocation and motor failure handling
- PX4 ROS 2 user guide
- PX4 flight log analysis, pyulog, PlotJuggler, Flight Review
- MuJoCo documentation and MuJoCo Python bindings

More papers

- Lanzon, Freddi, Longhi, Flight control of a quadrotor vehicle subsequent to a rotor failure, JGCD 2014
- Lippiello, Ruggiero, Serra, Emergency landing for a quadrotor in case of a propeller failure (PID and backstepping), SSRR and IROS 2014
- Sun et al., Incremental nonlinear fault-tolerant control of a quadrotor with complete loss of two opposing rotors, IEEE T-RO 2020
- Sun, Cioffi, de Visser, Scaramuzza, Autonomous quadrotor flight despite rotor failure with onboard vision sensors, IEEE RA-L 2021

- Ke, Cai, Quan, Uniform passive fault-tolerant control of a quadcopter with one, two, or three rotor failure, IEEE T-RO 2023
- Zhang and Jiang, Bibliographical review on reconfigurable fault-tolerant control systems, Annual Reviews in Control 2008
- Freddi, Longhi, Monteriu, Actuator fault detection system for a mini-quadrotor, ISIE 2010
- Amoozgar, Chamseddine, Zhang, Two-stage Kalman filter for actuator fault detection of a quadrotor, JIRS 2013
- ML fault detection, isolation and severity prediction for VTOL, 2026
- Active propeller fault detection and isolation via vibration model, 2026
- UAV-SEAD state estimation anomaly dataset (PX4, includes motor failure), 2026
- Sudheer and Varanasi (IIT Jodhpur), Adaptive fault tolerant control of quadrotors, 2026
- Maurya, Patel, Das, Maity (IIT Bombay), Fault-tolerant control for actuator failure in an octorotor UAV, 2025

More code and simulators

- iff-gsc/FTC_Quadrotor_EuroGNC_2022: passive FTC for up to three successive rotor failures
- Tianxing-Chen/uav_ftc
- pranshuk22/quadrotor-rotor-loss-recovery: Koopman MPC study
- acados and CasADi for MPC
- Flightmare quadrotor simulator
- Stable-Baselines3 for RL (PPO, SAC)

Hardware (Track B)

- eCalc multicopter calculator for thrust, current and flight time
- Motor manufacturers' thrust tables (for example T-Motor, iFlight, EMAX)
- Holybro Pixhawk 6C and the PX4 supported autopilots list